Designing the Data Management Layer with H2

This lab is individual homework. Work through it at home, then submit the URL of your GitHub repository and your LAB_REPORT.md on MyCourses before Sunday 8 November 2026 at 23:55.

You will move ParcelPoint, the campus parcel locker program, from a store in memory to H2, a database that runs inside the program. Part 0 starts from building the program and adds one idea at a time, and every command comes with an explanation of what it does. Part A asks you to model the data management layer with a package diagram, a class diagram and a sequence diagram. Part B asks you to explain the design decisions behind it. Section 5 shows how each step carries over to the data store of your Project Phase 2 team.

1. Learning outcomes

This lab trains six skills.

1. Explain why data kept in the memory of a program disappears when the program ends, and how a data store keeps it.
2. Change the data store of a program in one place, behind a repository interface.
3. Read the rows that a program stored, with the H2 Shell and simple SQL statements.
4. Write a JDBC method with a PreparedStatement and try-with-resources, following an existing method.
5. Model the data management layer with a package diagram, a class diagram and a sequence diagram in Mermaid.
6. Choose a data store from the requirements of a project, and justify the choice.

2. Before you start

Tools	JDK 17 or newer, Maven, Git, a GitHub account, and Visual Studio Code with the Extension Pack for Java, all from Part A of the Code Quality and Design Quality lab
Starter	The starter zip of this lab on MyCourses, which holds the ParcelPoint project and LAB_REPORT.md
Reference	H2 tutorial, https://h2database.com/html/tutorial.html, and the JDBC tutorial, https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html

2.1 Check the tools

In this sheet, a terminal means Command Prompt or PowerShell on Windows, and the Terminal app on macOS. Open a terminal, and run the three commands below one at a time.

```
java -version
mvn -version
git --version
```

The first command must show version 17 or newer. The second must show the version of Maven and, a few lines below it, Java version 17 or newer. The third must show a version number. If a command is not recognised, or shows a Java version below 17, repeat Part A of the Code Quality and Design Quality lab for that tool, then open a new terminal.

Open VS Code, click View, then Extensions, and check that Extension Pack for Java by Microsoft is installed. If it is not, type `Extension Pack for Java` in the search box, click the result whose publisher is Microsoft, and click Install.

2.2 Create your repository

Keep all your work for this lab in one public GitHub repository named `itcs382-h2-lab`. Anyone can read a public repository, so do not write your name or student ID anywhere in it. MyCourses links your submission to you.

1. Sign in at https://github.com, click the + button at the top right, and click New repository. In Repository name, type `itcs382-h2-lab`. Choose Public, leave every other option unchanged, and click Create repository.
2. On the page that opens, find the Quick setup box, make sure that HTTPS is selected, and click the copy button next to the address, which ends in .git.
3. In VS Code, click View, then Command Palette, type `Git: Clone`, and press Enter. Paste the address, and press Enter.
4. Choose a folder that OneDrive, iCloud Drive or Google Drive does not sync, such as C:\projects on Windows or a folder named projects in your home folder on macOS, because syncing can interfere with the files that Git, Maven and H2 write. Create the folder with the New Folder button of the dialog if it does not exist, and click Select as Repository Destination. Sign in to GitHub in the browser if VS Code asks you to.
5. When VS Code asks whether to open the cloned repository, click Open. If it asks whether you trust the authors of the files in this folder, click Yes, I trust the authors.
6. Download the starter zip from MyCourses and unzip it. Inside it is a folder named ITCS382_Lab_H2_Data_Store. Open that folder in File Explorer on Windows, or in Finder on macOS. On macOS, press Command+Shift+. first, so that Finder also shows the file .gitignore, whose name starts with a dot.
7. Copy the three items inside, the folder parcelpoint-h2, the file LAB_REPORT.md and the file .gitignore, and paste them into your repository folder. That folder has the name of your repository and sits in the folder that you chose in item 4.
8. In VS Code, check that the Explorer on the left shows the three items.

The file .gitignore tells Git not to save the folders target and data, which Maven and H2 create on your computer. If it is missing, right-click an empty area of the Explorer, click New File, name it .gitignore, type the two lines below, and save it with Ctrl+S, or Command+S on macOS.

```
target/
data/
```

2.3 Commit your work

A commit records the current state of your files, and a sync sends your commits to GitHub. Commit at the points that the steps name. Make the first commit now, before you change anything.

1. Click View, then Source Control. Type `Starter code` in the Message box, and click Commit. If VS Code asks whether to stage all your changes and commit them directly, click Yes.
2. Click Publish Branch or Sync Changes, whichever button VS Code shows.
3. Click Terminal, then New Terminal, and run the command below. It prints the last commit on one line, which starts with a code of seven letters and digits, such as 3f9c2ab.

```
git log --oneline -1
```

Open LAB_REPORT.md, write that code on the line Commit of the starter, and save the file. Steps 0.6 and 0.11 ask for the code of their commits in the same way. Then open your repository on GitHub in the browser, and check that the files are there.

2.4 Level of AI usage

This lab is at Level 3, AI Collaboration. You may use an AI tool in any part of this lab. Record the tool, the exact model and the date of every use in the AI usage declaration at the end of the report. Check every AI answer yourself, because an AI can be wrong, and explain in the report how you checked it.

Part 0: From a store in memory to H2, one step at a time

ParcelPoint runs the parcel lockers on a university campus. Staff store each parcel in one of three lockers, whose sizes are SMALL, MEDIUM and LARGE, and students pick up their parcels and pay a fee for every day after the third. The program has three layers in four Java packages. The package cli reads what the user types. The packages service and domain form the business layer, which decides where a parcel goes and what it costs. The package storage is the data management layer, which keeps the parcels. Main, in the package parcelpoint, creates the objects and connects them. Steps 0.1 to 0.5 run the program with a store in memory and show how the layers meet. Steps 0.6 to 0.8 move the parcels into H2 and look inside the database, and Steps 0.9 to 0.13 finish the H2 repository.

Step 0.1: Build the program GUIDED

Every command in Part 0 runs in the VS Code terminal. The terminal opens in your repository folder, so first move into the project folder.

```
cd parcelpoint-h2
```

`cd` changes the folder of the terminal. The folder parcelpoint-h2 holds pom.xml, the file that tells Maven what the project needs. Now build the project.

```
mvn package
```

Maven reads pom.xml, downloads the H2 library and the Maven plugins the first time, compiles the Java files, and packs the classes into a jar file. The first run can take a few minutes. Near the end, the output must show this line.

```
[INFO] BUILD SUCCESS
```

Maven writes everything that it builds into the folder target. Open it in the Explorer. It holds parcelpoint-h2-1.0.jar, which is the program, and the folder lib with h2-2.5.252.jar, which is the H2 library. pom.xml lists H2 as the one library that the project needs, and Maven copies it into lib for you.

While Maven works, VS Code reads the project too. Wait until the status bar at the bottom of the window shows Java: Ready. If it shows Java: Lightweight Mode, click View, then Command Palette, run `Java: Switch to Standard Mode`, and wait for Java: Ready. VS Code then marks mistakes in the Java files as you type, which helps in Step 0.11.

Step 0.2: Store two parcels in memory GUIDED

Start the program.

```
java -jar target/parcelpoint-h2-1.0.jar
```

`java -jar` runs the program inside the jar file. ParcelPoint prints its five commands and waits after the > sign.

```
ParcelPoint commands:
  store <code> <studentId> <SMALL|MEDIUM|LARGE>
  pickup <code> <days>
  find <code>
  staff
  quit
>
```

The words in angle brackets are the parts that you fill in. Type the four lines below after the > sign, and press Enter after each one.

```
store P1001 S1001 SMALL
store P1002 S1002 LARGE
staff
quit
```

The screen then looks like this. Each line that starts with > holds a command that you typed, and the lines under it are the answer of the program.

```
> store P1001 S1001 SMALL
Parcel P1001 is in locker 1
> store P1002 S1002 LARGE
Parcel P1002 is in locker 3
> staff
P1001, S1001, SMALL, STORED, locker 1
P1002, S1002, LARGE, STORED, locker 3
Free lockers: 1
> quit
```

`store` takes the code of a parcel, the ID of its student and its size, and puts the parcel into the first free locker that fits it. Locker 1 is SMALL, locker 2 is MEDIUM and locker 3 is LARGE, so the large parcel goes to locker 3. `staff` lists every parcel with its status and its locker, then counts the free lockers. `quit` ends the program.

Step 0.3: Run the program again GUIDED

Before you run the program again, predict what `staff` will show this time, and write your prediction under 0.3 in the report. Then start the program, and type `staff` and `quit`.

```
java -jar target/parcelpoint-h2-1.0.jar
```

This time the answer is short.

```
> staff
Free lockers: 3
> quit
```

Both parcels are gone, and all three lockers are free. Copy this output into the report, and write in one sentence where you think the parcels of the first run were kept. The next two steps show whether you were right.

Step 0.4: The repository interface GUIDED

In the Explorer, open the folder parcelpoint-h2/src/main/java/parcelpoint. Each folder inside it is one Java package, namely cli, service, domain and storage, and Main.java sits in the package parcelpoint

itself. Open ParcelRepository.java in the folder storage. Below its package and import lines, it holds this interface.

```
public interface ParcelRepository {
    void save(Parcel parcel);

    Optional<Parcel> findByCode(String code);

    List<Parcel> findAll();
}
```

An interface lists operations without their code. ParcelRepository has three. `save` keeps a parcel, `findByCode` returns the parcel with a given code, and `findAll` returns every parcel. The rest of the program reaches the parcels through these three operations alone, so this interface is the boundary of the data management layer. Above it, the code works with Parcel objects. Below it, a class decides where the parcels really go.

Now open InMemoryParcelRepository.java, the class that the program uses today. It starts like this.

```
public class InMemoryParcelRepository implements ParcelRepository {
    private final Map<String, Parcel> parcels = new LinkedHashMap<>();

    @Override
    public void save(Parcel parcel) {
        parcels.put(parcel.getCode(), parcel);
    }
```

`implements ParcelRepository` promises the three operations of the interface. The class keeps the parcels in a map from each code to its parcel, a Java object that lives in the memory of the running program. When the program ends, that memory is freed and the map is gone, which is why Step 0.3 found no parcels. A program needs a data store, such as a file or a database, to keep its data from one run to the next.

Step 0.5: Where the program chooses its store GUIDED

Open ParcelService.java in the folder service. Find its field `repository` and its constructor, which start like this.

```
private final ParcelRepository repository;

public ParcelService(ParcelRepository repository) {
    this.repository = repository;
```

ParcelService keeps a field whose type is the interface ParcelRepository, and it receives the object for that field through its constructor. It never creates a repository itself, so it does not know which class keeps the parcels. Now find the method `store` a few lines below. After it puts the parcel into a locker, it hands the parcel to the repository.

```
locker.setParcel(parcel);
parcel.storeIn(locker.getNumber());
repository.save(parcel);
```

Last, open Main.java, which holds the method `main`.

```
public static void main(String[] args) {
    ParcelRepository repository = new InMemoryParcelRepository();
    ParcelService service = new ParcelService(repository);
    new ParcelCommandLine(service).run(new Scanner(System.in));
}
```

Main is the one class that names a concrete repository. It creates an InMemoryParcelRepository, passes it to ParcelService, passes the service to ParcelCommandLine, and starts the command line. This follows the Dependency Inversion Principle. ParcelService depends on the interface, and Main decides which class stands behind it. Write under 0.5 in the report the line of Main that you would change to keep the parcels somewhere else.

Step 0.6: Switch to H2 GUIDED

H2 is a database written in Java. It runs inside your program as a library, keeps its data in a file and needs no separate server, so it is called an embedded database. The starter already holds H2ParcelRepository.java, a second class in the folder storage that implements ParcelRepository with H2. To use it, change two lines of Main.java. Find the line below.

```
import parcelpoint.storage.InMemoryParcelRepository;
```

Change it to this line.

```
import parcelpoint.storage.H2ParcelRepository;
```

Then find this line in the method `main`.

```
ParcelRepository repository = new InMemoryParcelRepository();
```

Change it to this line, and save the file with Ctrl+S, or Command+S on macOS.

```
ParcelRepository repository = new H2ParcelRepository("./data/parcelpoint");
```

The text in quotes tells H2 where to keep its database. `./data/parcelpoint` names a database called parcelpoint in the folder data inside the current folder, and H2 stores it in the file parcelpoint.mv.db. Build the program again and start it.

```
mvn package
java -jar target/parcelpoint-h2-1.0.jar
```

Type the four lines of Step 0.2 again. The screen looks like this.

```
> store P1001 S1001 SMALL
Parcel P1001 is in locker 1
> store P1002 S1002 LARGE
Parcel P1002 is in locker 3
> staff
Free lockers: 1
> quit
```

`store` works as before, but `staff` lists no parcels, although it counts one free locker. Step 0.9 finds out why. Look at the Explorer. A new folder named data holds the file parcelpoint.mv.db, which H2 created on the first run. ParcelService, ParcelCommandLine and the classes in domain did not change at all. Commit with the message `Switch to H2`, sync, and write the code of the commit under 0.6 in the report, as in Section 2.3.

Step 0.7: Look inside the database with the H2 Shell GUIDED

The H2 library also holds the H2 Shell, a small tool that runs SQL statements on a database from the terminal. The Shell needs four options, and the starter keeps them in the file h2shell.txt in the folder parcelpoint-h2, one option on each line. Open it.

```
-cp target/lib/h2-2.5.252.jar
org.h2.tools.Shell
-url jdbc:h2:./data/parcelpoint
-user sa
```

The first line tells Java where to find the H2 library. The second names the class to run, which is the H2 Shell. The third gives the database as a JDBC URL, in which `jdbc:h2:` says that H2 opens it and `./data/parcelpoint` is the database that ParcelPoint uses. The last line gives the user name sa, the same as in H2ParcelRepository, with an empty password. Start the Shell with the command below.

```
java "@h2shell.txt"
```

The @ in front of the file name tells Java to read its options from the file, and the quotes stop PowerShell from treating the @ as part of its own syntax. The Shell prints a welcome line and a short list of its own commands, then waits after the prompt sql>. Type the statement below after sql>, and press Enter. Every SQL statement ends with a semicolon.

```
SELECT * FROM PARCEL;
```

H2 answers with the rows of the table.

```
sql> SELECT * FROM PARCEL;
CODE  | STUDENT_ID | PARCEL_SIZE | STATUS | LOCKER_NUMBER
P1001 | S1001      | SMALL       | STORED | 1
P1002 | S1002      | LARGE       | STORED | 3
(2 rows, 7 ms)
```

`SELECT * FROM PARCEL` reads every column of every row of the table PARCEL. Each row holds one Parcel object, and each column holds one of its fields. The size and the status are stored as their names, such as SMALL and STORED. The time in the last line differs on your computer. Copy this output under 0.7 in the report. Next, ask H2 how it defined the table.

```
SHOW COLUMNS FROM PARCEL;
```

H2 lists one row for each column of the table.

```
sql> SHOW COLUMNS FROM PARCEL;
FIELD         | TYPE                   | NULL | KEY | DEFAULT
CODE          | CHARACTER VARYING(20) | NO   | PRI | NULL
STUDENT_ID    | CHARACTER VARYING(20) | NO   |     | NULL
PARCEL_SIZE   | CHARACTER VARYING(10) | NO   |     | NULL
STATUS        | CHARACTER VARYING(10) | NO   |     | NULL
LOCKER_NUMBER | INTEGER                | YES  |     | NULL
(5 rows, 9 ms)
```

H2ParcelRepository created this table when ParcelPoint first started with H2, with the statement below at the top of H2ParcelRepository.java. H2 shows the type VARCHAR under its full SQL name, CHARACTER VARYING.

```
private static final String CREATE_TABLE = """
        CREATE TABLE IF NOT EXISTS PARCEL (
            CODE          VARCHAR(20) PRIMARY KEY,
            STUDENT_ID    VARCHAR(20) NOT NULL,
            PARCEL_SIZE   VARCHAR(10) NOT NULL,
            STATUS        VARCHAR(10) NOT NULL,
            LOCKER_NUMBER INT
        )""";
```

`IF NOT EXISTS` makes H2 create the table on the first run and leave it alone after that, so the program needs no set-up before its first run. CODE is the primary key, marked PRI, so no two rows can have the same code. LOCKER_NUMBER may be NULL, an empty value, because a parcel that was picked up has no locker. Last, ask for the parcels that still wait in a locker.

```
SELECT CODE, LOCKER_NUMBER FROM PARCEL WHERE STATUS = 'STORED';
```

This time H2 returns two columns, and the rows that match the condition after WHERE.

```
sql> SELECT CODE, LOCKER_NUMBER FROM PARCEL WHERE STATUS = 'STORED';
CODE  | LOCKER_NUMBER
P1001 | 1
P1002 | 3
(2 rows, 0 ms)
```

Keep the Shell open for the next step.

Step 0.8: One program at a time GUIDED

Leave the Shell running in the first terminal. Open a second terminal with Terminal, then New Terminal. VS Code lists both terminals at the right of the terminal panel, and you can click either one to switch to it. In the second terminal, move into the project folder and start ParcelPoint.

```
cd parcelpoint-h2
java -jar target/parcelpoint-h2-1.0.jar
```

The program stops at once with a long error. Its first line ends with `Cannot create the PARCEL table`. A few lines below, a line that starts with `Caused by` says `Database may be already in use`.

The Shell in the first terminal still holds the database open. An embedded database belongs to the program that opened it, so H2 locks the file parcelpoint.mv.db, and a second program cannot open it at the same time. The first line of the error comes from ParcelPoint, not from H2. Answer question 0.8 in the report, which asks why ParcelPoint could not start and which class wrote that first line.

Then click the first terminal, and type `quit` after sql> to close the Shell. Run ParcelPoint again in the second terminal. This time it starts, so type `quit`. Keep working in the second terminal from now on.

Step 0.9: The parcels are in H2, but staff cannot see them YOUR TURN

Start ParcelPoint, and type the four lines below.

```
staff
find P1001
find P1002
quit
```

The screen looks like this.

```
> staff
Free lockers: 3
> find P1001
P1001, S1001, SMALL, STORED, locker 1
> find P1002
P1002, S1002, LARGE, STORED, locker 3
> quit
```

`find` reads one parcel from H2 and shows it, so both parcels are in the database. `staff` shows none of them and even counts all three lockers as free. Open H2ParcelRepository.java and compare the method `findByCode`, which `find` uses, with the method `findAll`, which `staff` uses. Then read the constructor of ParcelService. Answer question 0.9 in the report. Explain why `staff` shows no parcel, and why all three lockers look free.

Step 0.10: Read findByCode line by line GUIDED

Before you write `findAll`, read the method that already works. This is `findByCode` in H2ParcelRepository.java.

```
@Override
public Optional<Parcel> findByCode(String code) {
    String sql = "SELECT * FROM PARCEL WHERE CODE = ?";
    try (Connection connection = connect();
         PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setString(1, code);
        try (ResultSet row = statement.executeQuery()) {
            return row.next() ? Optional.of(toParcel(row)) : Optional.empty();
        }
    } catch (SQLException e) {
        throw new IllegalStateException("Cannot read parcel " + code, e);
    }
}
```

The method takes four steps, and every method that reads the database in this lab takes the same four.

1. `connect()` opens a connection to the database. Its code sits above the method `save`, and it uses the JDBC URL, the user sa and an empty password.
2. `connection.prepareStatement(sql)` prepares the SQL statement. The ? is a placeholder for a value, and `statement.setString(1, code)` puts the code into the first placeholder.
3. `statement.executeQuery()` runs the statement and returns a ResultSet, which stands just before the first row of the answer. `row.next()` moves to the next row, and it returns false when no row is left.
4. `toParcel(row)` reads the columns of the current row and builds a Parcel object from them.

The `try (...)` around the connection, the statement and the ResultSet is called try-with-resources. Java closes each of them when the block ends, even when an exception is thrown, so the method never leaves a connection open. If any step fails, the `catch` block turns the SQLException into an IllegalStateException with a clear message, as the constructor did in Step 0.8. The placeholder also keeps the code that a user typed out of the SQL text, and the optional Step 0.13 shows why that matters.

Now look at `toParcel`, at the end of the file.

```
private Parcel toParcel(ResultSet row) throws SQLException {
    return new Parcel(
            row.getString("CODE"),
            row.getString("STUDENT_ID"),
            ParcelSize.valueOf(row.getString("PARCEL_SIZE")),
            ParcelStatus.valueOf(row.getString("STATUS")),
            row.getObject("LOCKER_NUMBER", Integer.class));
}
```

`row.getString("CODE")` reads the column CODE of the current row. `ParcelSize.valueOf` turns the text SMALL back into the value ParcelSize.SMALL, which works because the table stores the name of the size. This one method holds the whole mapping between a row of PARCEL and a Parcel object, so a change to the table means a change here and nowhere else. Question 2 of Part B comes back to this mapping.

Step 0.11: Write findAll YOUR TURN

Write `findAll` so that `staff` lists every parcel. Follow the four steps of `findByCode`.

- Use the SQL statement `SELECT * FROM PARCEL ORDER BY CODE`, which reads every row in the order of the codes. It has no placeholder, so it needs no `setString`.
- Create an empty list before the `try`, with the line below.

```
List<Parcel> parcels = new ArrayList<>();
```

- Inside the `try`, loop with `while (row.next())`, turn each row into a Parcel with `toParcel`, and add it to the list.
- Catch SQLException as `findByCode` does, with the message `Cannot read the parcels`, and return the list after the `try`.
- Delete the two comment lines that start with `// TODO`.

Build the program and start it, then type `staff` and `quit`.

```
mvn package
java -jar target/parcelpoint-h2-1.0.jar
```

When `findAll` works, the screen looks like this.

```
> staff
P1001, S1001, SMALL, STORED, locker 1
P1002, S1002, LARGE, STORED, locker 3
Free lockers: 1
> quit
```

Both parcels are back, and so are their lockers, because the constructor of ParcelService puts every waiting parcel from `findAll` back into its locker. If `mvn package` reports an error, read the line that names the file and the line number, and compare your code with `findByCode`. Copy your `findAll` method into the report. Commit with the message `Write findAll`, sync, and write the code of the commit under 0.11 in the report.

Step 0.12: Pick up a parcel GUIDED

Start the program, and type the three lines below.

```
pickup P1001 5
staff
quit
```

The screen looks like this.

```
> pickup P1001 5
Parcel P1001 picked up. Fee: 20.0 baht
> staff
P1001, S1001, SMALL, PICKED_UP
P1002, S1002, LARGE, STORED, locker 3
Free lockers: 2
> quit
```

Parcel P1001 waited 5 days, which is 2 days more than the 3 free days, and a SMALL parcel costs 10 baht for each extra day, so the fee is 20.0 baht. `pickup` marks the parcel as PICKED_UP, empties its locker, and calls `save` again. Open the H2 Shell, and type `SELECT * FROM PARCEL;` after sql>, as in Step 0.7.

```
java "@h2shell.txt"
```

The table looks like this now.

```
sql> SELECT * FROM PARCEL;
CODE  | STUDENT_ID | PARCEL_SIZE | STATUS    | LOCKER_NUMBER
P1001 | S1001      | SMALL       | PICKED_UP | null
P1002 | S1002      | LARGE       | STORED    | 3
(2 rows, 7 ms)
```

The table still has two rows, and the row of P1001 has changed. `save` uses the statement `MERGE INTO PARCEL KEY (CODE)`, which updates the row whose CODE matches, or inserts a new row when no row matches. Copy this output under 0.12 in the report, then type `quit` to close the Shell.

Step 0.13: What the placeholder prevents OPTIONAL

This step breaks `findByCode` on purpose for a few minutes, then repairs it. Commit your work first. In `findByCode`, find the line below.

```
String sql = "SELECT * FROM PARCEL WHERE CODE = ?";
```

Change it to this line.

```
String sql = "SELECT * FROM PARCEL WHERE CODE = '" + code + "'";
```

Then delete the line `statement.setString(1, code);`, because the statement no longer has a placeholder. The SQL is now joined together from strings, with the code that the user typed in the middle. Build the program and start it.

```
mvn package
java -jar target/parcelpoint-h2-1.0.jar
```

Type the three lines below. The second line has no spaces.

```
find P9999
find P9999'OR'1'='1
quit
```

The screen looks like this.

```
> find P9999
No parcel with code P9999
> find P9999'OR'1'='1
P1001, S1001, SMALL, PICKED_UP
> quit
```

No parcel has the code `P9999'OR'1'='1`, but the program shows one. The quotes that you typed closed the text in the SQL statement and added a condition, so H2 ran `SELECT * FROM PARCEL WHERE CODE = 'P9999'OR'1'='1'`. The condition `'1'='1'` is true for every row, so the query returned all rows, and `findByCode` showed the first one. This attack is called SQL injection, and a user who types the same trick into a real system can read data that is not theirs.

Repair the method now. Put back the original line with the ?, and put the line below back right after the line that calls `prepareStatement`.

```
statement.setString(1, code);
```

Build and start the program again, then type `find P9999'OR'1'='1` and `quit`.

```
> find P9999'OR'1'='1
No parcel with code P9999'OR'1'='1
> quit
```

With the placeholder, H2 compares CODE with the whole text that you typed, so no row matches. Write under 0.13 in the report what the two runs showed. If Source Control still lists H2ParcelRepository.java as changed, right-click it there, click Discard Changes, and click Discard File to confirm. This returns the file to your last commit.

3. The ideas you are about to practice

3.1 The data management layer

The data management layer translates between the objects of the program and its data store. The rest of the program works with Parcel objects and never sees SQL, a file or a connection. A repository interface for each kind of object marks the boundary of the layer. The interface belongs to the design, and each class that implements it belongs to one data store. Main creates the class that the program uses, so the data store changes in one place, as in Step 0.6. The same design lets you test the business layer against the store in memory, with no database at all.

3.2 Three diagrams of one layer

A package diagram shows which package may use which. Its arrows point from the package that uses to the package that is used, and the Phase 2 package diagram marks the package that accesses the data store. A class diagram of the data management layer shows the repository interface, the classes that implement it and the classes that use it. A sequence diagram shows the messages of one command in the order in which they happen, from the user down to the repository.

3.3 Connascence between the code and the data

Two parts of a program are connascent when a change to one forces a change to the other. The PARCEL table and the enumeration ParcelSize are connascent, because the table must hold values that `ParcelSize.valueOf` can turn back into sizes. Connascence of name means that both sides must agree on a name. Connascence of position means that they must agree on an order, and connascence of convention means that they must agree on what a value stands for. A weaker form of connascence is easier to see and safer to change.

Part A: Model the design

Step A1: Package dependencies GUIDED

One package depends on another when a class of the first uses a class of the second. The import lines at the top of each Java file show these uses, and classes of the same package need no import. For example, ParcelCommandLine.java in the package cli has these imports.

```
import parcelpoint.domain.Locker;
import parcelpoint.domain.Parcel;
import parcelpoint.domain.ParcelSize;
import parcelpoint.service.ParcelService;
```

cli therefore depends on domain and on service, which gives the two arrows below.

```
cli --> domain
cli --> service
```

Imports from java.util or java.sql name no package of ParcelPoint, so they give no arrow. Open the other Java files in the same way, and complete the flowchart under A1 in the report with one arrow for

each dependency, from the package that uses to the package that is used. Mark the package that accesses the data store, as the Phase 2 package diagram must, for example with the label below.

```
storage["storage, accesses the data store"]
```

To see the diagram drawn, open https://mermaid.live in a web browser, and paste the Mermaid code into the Code box on the left. GitHub also draws the diagrams in LAB_REPORT.md after you sync. When you finish, answer the question under the flowchart, which asks whether cli depends on storage.

Step A2: The class diagram of the data management layer YOUR TURN

Complete the class diagram under A2 in the report. Show ParcelService, the interface ParcelRepository with its three operations, and the two classes that implement it, InMemoryParcelRepository and H2ParcelRepository. Draw the association from ParcelService to the interface, and a realization from each class to the interface. Mermaid writes a realization as below, and it writes a generic type such as Optional<Parcel> as `Optional~Parcel~`.

```
ParcelRepository <|.. H2ParcelRepository
```

Check the drawing on mermaid.live before you commit. ParcelService must have no line to H2ParcelRepository, because the code of ParcelService never names that class.

Step A3: A sequence diagram of one command YOUR TURN

Draw a sequence diagram under A3 in the report for the command `store P1003 S1003 MEDIUM`, from the staff member who types it down to the repository. Follow the code, as you did in Step 0.5. ParcelCommandLine calls `store` on ParcelService. ParcelService asks each Locker whether the parcel fits, puts the parcel into the first free locker that fits, tells the parcel its locker number, and saves it. Show each message as an operation of the class that receives it, with a loop around the calls to `fits`. The report already holds the first lines.

```
sequenceDiagram
    actor Staff
    participant cli as :ParcelCommandLine
    Staff->>cli: store P1003 S1003 MEDIUM
```

Add a participant for each object that receives a message, and end the diagram at the interface ParcelRepository. Then answer the question under the diagram, which asks why the last lifeline is the interface rather than H2ParcelRepository.

Part B: Design questions

Answer these questions in the report, in two to four sentences each.

1. Identify the files that you changed to move from the store in memory to H2, and explain why no other class had to change. Use the terms of this course.
2. The PARCEL table stores the size of a parcel as SMALL, MEDIUM or LARGE rather than as 1, 2 or 3. Identify the type of connascence between ParcelSize and the table in each case, and explain which one is weaker.
3. Project Phase 2 asks for a data store that needs no separate server. Explain why an embedded database such as H2 meets this rule, and describe the limit of an embedded database that Step 0.8 showed.
4. State the data store that you would choose for your Project Phase 2 team, files or an embedded database, the requirement that drives the choice, and the package that will access it.

4. Deliverables

Before the last commit, complete the report.

1. Fill the line Repository URL and the Tools table at the top of the report. Take the JDK and Maven versions from Section 2.1. For VS Code, click Help, then About on Windows, or Code, then About Visual Studio Code on macOS.
2. Fill the AI usage declaration at the end of the report, or write None in its first row if you used no AI tool.
3. Commit and sync. Open your repository on GitHub, and check that it shows parcelpoint-h2 and LAB_REPORT.md, and no target or data folder. Then open its URL in a private or incognito window of your browser, where you are not signed in, to check that anyone can see it.
4. On MyCourses, open the lab assignment, paste the URL of your repository, upload LAB_REPORT.md, and submit before Sunday 8 November 2026 at 23:55.

Your repository should look like this.

```
itcs382-h2-lab/
  .gitignore
  LAB_REPORT.md
  parcelpoint-h2/
    .gitignore
    README.md
    h2shell.txt
    pom.xml
    src/main/java/parcelpoint/
      Main.java
      cli/ParcelCommandLine.java
      domain/Locker.java
      domain/Parcel.java
      domain/ParcelSize.java
      domain/ParcelStatus.java
      service/ParcelService.java
      storage/H2ParcelRepository.java
      storage/InMemoryParcelRepository.java
      storage/ParcelRepository.java
```

This is individual work, so copying another student's report or code counts as plagiarism. Report any problem with the tools to the instructor before the deadline.

5. From this lab to Project Phase 2

The data store of Project Phase 2 follows the same rules. The table shows where this lab practised each one.

Project Phase 2 asks for	Where you practised it
Data that stays after the program ends, in files or in an embedded database such as H2	Steps 0.3 and 0.6
A data store that needs no separate server, with files or tables that the program creates on its first run	Steps 0.6 and 0.7, with `CREATE TABLE IF NOT EXISTS`
At most one external library, and only for the data store	pom.xml, which lists H2 as its one dependency

Project Phase 2 asks for	Where you practised it
A package diagram that marks the package that accesses the data store	Step A1
A design that keeps the rest of the code independent of the data store, explained in the report	Steps 0.5 and 0.6, and question 1 of Part B
A justified choice of data store	Question 4 of Part B

A team that chooses files follows the same design. Its repository class implements the same kind of interface, reads and writes a file instead of a database, and Main creates it. Classes such as ParcelService stay the same whichever store the team chooses.

6. If something goes wrong

- If the terminal says that java, mvn or git is not recognised, close the terminal and open a new one, or restart VS Code, so that it reads the new settings. If the message stays, repeat the matching step of Part A of the Code Quality and Design Quality lab.
- If `mvn -version` or `mvn package` reports a Java version below 17, Maven is still using an older JDK. Run the JDK installer again, choose Set JAVA_HOME variable on Windows, and open a new terminal.
- If `mvn package` cannot download H2 or a plugin, check the internet connection, and run `mvn package` again.
- If `java "@h2shell.txt"` prints an error that starts with `Error: could not open`, the terminal is not in the folder parcelpoint-h2. Run `cd parcelpoint-h2` first.
- If the Shell answers `Table "PARCEL" not found (this database is empty)`, ParcelPoint has never started with H2 in this folder. Type `quit`, finish Step 0.6, and start the Shell again.
- If ParcelPoint or the Shell reports `Database may be already in use`, another program still has the database open, such as the H2 Shell or ParcelPoint in another terminal. Close it as in Step 0.8, and run the command again.
- If `staff` shows parcels that you did not store, delete the folder data in the Explorer while no program runs, and repeat Step 0.6 from the `java` command.
- If the status bar shows Java: Error or Java: No Projects Imported, click View, then Command Palette, run `Java: Clean Java Language Server Workspace`, and confirm the restart.
- If Publish Branch or Sync Changes asks you to sign in, sign in to GitHub in the browser, and allow VS Code to use your account.